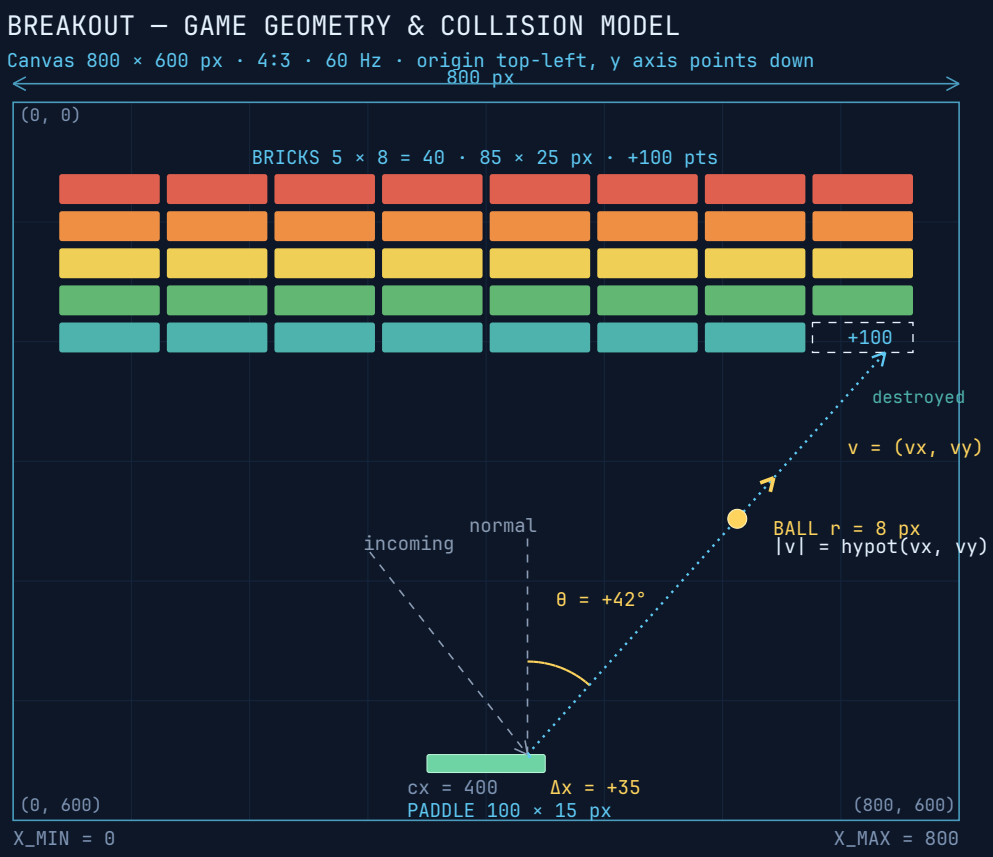
دانشگاه تهران
دانشکده علوم کامپیوتر

برنامه‌سازی پیشرفته

فاز پروژه

۰۱ هسته بازی Breakout

اهداف، نیازمندی‌های فنی، طراحی شی‌گرا و معیارهای ارزیابی




```
CANVAS
size    800 × 600 px (4:3)
tick    60 Hz
origin  top-left, y grows down

BRICKS
grid    5 rows × 8 cols = 40
size    85 × 25 px, gap 6
reward  +100 pts each

PADDLE
size    100 × 15 px
pos     y = 545, x in [0, 700]

BALL
radius  r = 8 px
speed   |v| = hypot(vx, vy)

PADDLE REFLECTION
offset = Δx / (w / 2)
clamped to [-1, 1]
θ = offset · 60°
vx = |v| · sin θ
vy = - |v| · cos θ
e.g. Δx = +35 → θ = +42°

CONTROLS
← →   move paddle
SPACE   serve
P       pause
R       reset
```

Dotted cyan = ball path after paddle hit · dashed grey = incoming path · yellow = velocity / angle

«در سال ۱۹۷۶، نولان باشنل در شرکت آتاری از استیو جابز خواست بازی Breakout را با کمترین تعداد تراشه سخت‌افزاری بسازد. جابز پروژه را به استیو وازنیاک سپرد؛ وازنیاک طی ۴ شبانه‌روز کار بی‌وقفه، مدار بازی را تنها با ۴۴ تراشه پیاده کرد. ابتکارات وازنیاک در بهینه‌سازی مدارهای گرافیکی این بازی، این تجربه را با طراحی نمایش تصویر و رنگ در رایانهٔ انقلابی Apple II مرتبط می‌دانند.»

فهرست مطالب

قوانین و شرایط تحویل ۲
معرفی بازی و اهداف ۳
بستر فنی و نسخهٔ جاوا ۴
اقدامات پیش از شروع ۵
ساختار و معماری سیستم ۶
موجودیت‌ها و هندسه بازی ۷
سازوکار گیم‌پلی و وضعیت‌ها ۸
گرافیک و واسط کاربری ۹
فیزیک و برخوردها ۱۰
گام‌های پیاده‌سازی ۱۲
معیارهای ارزیابی و ارائه ۱۳

قوانین و شرایط تحویل

این پروژه فرصتی است تا مفاهیم شی‌گرایی را در یک نرم‌افزار واقعی به کار ببرید. برای اینکه کار برای همه منصفانه باشد، چند قاعدهٔ ساده را با هم مرور می‌کنیم:

استقلال در پیاده‌سازی و مالکیت کد

این پروژه **فردی** است. کدی که تحویل می‌دهید باید حاصل کار خودتان باشد. منابع، راهنماها و مقاله‌ها را برای یادگیری بخوانید، اما کپی‌کردن کد دیگران یا تحویل کدی که خودتان آن را نمی‌فهمید، با هدف درس مغایرت دارد. در جلسه ارائه باید بتوانید دلیل هر بخش از کد را توضیح دهید.

ثبت پیوسته در گیت‌هاب

پیشرفت شما از روی کامیت‌های مرحله‌به‌مرحله[۱] در مخزن گیت‌هاب[۲] ارزیابی می‌شود. قابلیت‌ها را یکی‌یکی اضافه کنید و هر تغییر را با پیام روشن ثبت کنید. از ارسال کل پروژه در یک کامیت، در ساعات پایانی مهلت، خودداری کنید.

جلسه ارائه و گفت‌وگوی فنی

پس از تحویل مخزن در سامانهٔ ایلرن، دستیاران آموزشی در یک جلسه کد شما را بررسی می‌کنند. در این جلسه تسلط شما بر معماری، کلاس‌ها و پیاده‌سازی سنجیده می‌شود. اگر تسلط کافی نباشد، نمرهٔ فنی طبق بخش «معیارهای ارزیابی و ارائه» تعدیل می‌شود.

بستر تحویل و موعد نهایی

نشانی مخزن گیت‌هاب را پیش از پایان مهلت، در سامانهٔ آموزشی ایلرن[۳] بارگذاری کنید. مخزن باید عمومی باشد یا دسترسی آن را برای تیم تدریس باز کرده باشید.

استفاده از ابزارهای هوش مصنوعی

برای فهم مفاهیم، یافتن خطا و مقایسهٔ راه‌حل‌ها می‌توانید از ابزارهای هوش مصنوعی کمک بگیرید. با این حال، طراحی و منطق بازی باید کار خودتان باشد و باید بتوانید همهٔ بخش‌های آن را توضیح دهید.

پرسش و راهنمایی

اگر در معماری یا پیاده‌سازی به مشکل یا ابهامی برخوردید، در تالار گفتگو یا جلسات رفع اشکال بپرسید.

---

[۱] Git Commits
[۲] GitHub
[۳] eLearn

معرفی بازی و اهداف

در سال ۱۹۷۶ شرکت آتاری بازی Breakout را منتشر کرد. استیو وازنیاک[۴] مدار سخت‌افزاری این بازی را تنها با ۴۴ تراشه منطقی ساخت. بنا بر روایت‌های رایج، این تجربه با طراحی تصویر و رنگ در رایانهٔ Apple II هم مرتبط است. Breakout برای تمرین مهندسی نرم‌افزار شی‌گرا مناسب است: یک سکو در پایین صفحه، توپی که از دیواره‌ها کمانه می‌کند و چند آجر. هر کدام از این اجزا در کد مسئولیت روشنی دارد.

اهداف فاز اول

در این فاز هستهٔ بازی را به زبان جاوا[۵] می‌سازید. این فاز بر مهارت‌های زیر تمرکز دارد:

۱. **تفکیک مسئولیت‌ها:** منطق بازی، یعنی ریاضیات و قوانین برخورد، را از نقاشی و نمایش گرافیکی جدا نگه دارید.[۶]
۲. **اصل تک‌مسئولیتی:** هر کلاس یک کار مشخص داشته باشد و از کلاس‌های همه‌کاره پرهیز کنید.[۷،۸]
۳. **کپسوله‌سازی:** متغیرهای وضعیت را پنهان کنید و تغییر آن‌ها را فقط از طریق متدهای روشن انجام دهید.[۹]
۴. **مدیریت رویدادها:** کلیدها را بخوانید و فریم‌ها را در حلقهٔ اصلی بازی پشت سر هم به‌روز کنید.[۱۰]
۵. **طراحی برای تغییر:** کلاس‌ها را طوری بسازید که افزودن ذخیرهٔ رکورد در فایل و پاورآپ‌ها در فاز بعد ساده باشد.

محدودهٔ این فاز

هدف این فاز ساختار درست کد، فیزیک دقیق برخوردها و گیم‌پلی روان است. منوی پیچیده، صدا و گرافیک سه‌بعدی لازم نیست. یک رابط بصری ساده و مرتب کافی است.

---

[۴] Steve Wozniak
[۵] Java
[۶] SoC: Separation of Concerns
[۷] God Class
[۸] SRP: Single Responsibility Principle
[۹] Encapsulation
[۱۰] Event-Driven

بستر فنی و نسخهٔ جاوا

پیاده‌سازی فاز اول باید با زبان جاوا[۱۱] و یک کتابخانهٔ گرافیکی باشد که در محیط درس اجرا شود. **مسیر رسمی درس:** ACM[۱۲] + Swing روی **Java 11** است. اگر از بستر دیگری استفاده می‌کنید، آن را در README ثبت کنید. این بسترها مجازند، اما دستیاران در رفع اشکال آن‌ها امکان کمتری دارند.

بستر	نسخهٔ جاوا	یادداشت
ACM + Swing — مسیر رسمی	11 LTS	از کلاس‌های `acm.program.GraphicsProgram` و `acm.graphics.*` استفاده کنید. فایل `acm.jar` را در classpath قرار دهید.
Swing خالص	11 تا 21	بدون ACM، با `JPanel`، `Timer` و نقاشی دستی کار کنید. بافر دوگانه[۱۳] را خودتان مدیریت می‌کنید.
JavaFX	17 یا 21	از `Canvas` یا `Pane` استفاده کنید. ماژول‌های JavaFX را خودتان به پروژه اضافه کنید.
Processing	17	از حالت Java با `setup` و `draw` استفاده کنید. معماری MVC را خودتان طراحی کنید.

Java 8 هم با همان `acm.jar` در بسیاری از محیط‌های درس کار می‌کند. JavaFX از JDK 11 به بعد جدا از JDK توزیع می‌شود و Processing 4 بر پایهٔ JDK 17 است.

آزادی در چیدمان و ظاهر

ابعاد، مختصات و رفتارهایی که در این سند آمده برای موجودیت‌ها و فیزیک الزامی‌اند. رنگ آجرها، فونت پیام‌ها، چیدمان نوار وضعیت، نماد جان‌ها و هر بخش بصری بدون عدد یا فرمول را آزادانه انتخاب کنید. فقط خوانایی و روانی ۶۰ هرتز باید حفظ شود.

[۱۱] Java
[۱۲] ACM: Java Task Force
[۱۳] Double Buffering

اقدامات پیش از شروع

پیش از نوشتن اولین خط کد جاوا، این سه بخش را انجام دهید:

۱. مخزن گیت‌هاب، گیت‌ایگنور و کامیت‌ها

پروژه را با مدیریت نسخه و ثبت تاریخچهٔ تغییرات شروع کنید. این بخش سه کار دارد:

ایجاد مخزن در گیت‌هاب GitHub Repository

یک مخزن جدید در گیت‌هاب بسازید. کد، تاریخچهٔ تغییرات و نسخهٔ نهایی تحویلی شما در همین مخزن قرار می‌گیرد.

پیکربندی گیت‌ایگنور `.gitignore`

فایل‌های خروجی کامپایل با پسوند `.class`، فایل‌های موقت سیستم‌عامل و تنظیمات محیط توسعه[۱۴] را در `.gitignore` قرار دهید تا مخزن تمیز بماند.

کامیت‌های معنادار و استاندارد Conventional Commits

هر تغییر را با یک پیشوند استاندارد ثبت کنید، مثل `feat:`، `fix:` یا `refactor:`. با این کار تاریخچهٔ پروژه قابل فهم می‌ماند.

۲. تجربه بازی و درک رفتار فیزیکی

چند دقیقه یک نسخهٔ اصلی Breakout بازی کنید و این سه ویژگی را دقت کنید:

زاویه کمانه توپ از سکو

اگر توپ به مرکز سکو بخورد، تقریباً عمودی به بالا برمی‌گردد. هرچه نقطهٔ برخورد به لبه‌ها نزدیک‌تر باشد، زاویهٔ بازگشت تندتر و افقی‌تر می‌شود.

روانی شبیه‌سازی در ۶۰ هرتز

سکو بلافاصله به کلیدها واکنش نشان می‌دهد و توپ بدون لرزش، مکث یا پرش روی صفحه حرکت می‌کند.

مکانیک اصابت و انهدام آجرها

آجری که توپ به آن بخورد فوراً حذف می‌شود، توپ بازتاب می‌کند و ۱۰۰ امتیاز به بازیکن اضافه می‌شود.

۳. مدل‌سازی شی‌گرا و نقشه کلاس‌ها

طراحی پیش از کدنویسی، زمان پیاده‌سازی و اشکال‌زدایی را کم می‌کند:

ترسیم نقشه کلاس‌ها — اختیاری UML Class Diagram

اگر دوست دارید، پیش از کدنویسی نمودار کلاس[۱۵] را روی کاغذ یا با نرم‌افزار بکشید: کلاس‌ها، فیلدهای خصوصی و متدهای عمومی را مشخص کنید. تحویل نمودار در این فاز الزامی نیست.

نقش در جلسه ارائه

نقشهٔ معماری، حتی کاغذی، در جلسهٔ ارائه نشان می‌دهد که مسئولیت‌ها را خوب تفکیک کرده‌اید. اما برای تحویل نمودار نمرهٔ جداگانه‌ای در نظر گرفته نشده است.

---

[۱۴] IDE: Integrated Development Environment

[۱۵] UML: Unified Modeling Language

ساختار و معماری سیستم

فرض کنید کل بازی در یک کلاس ۲۵۰۰ خطی نوشته شده باشد: سرعت توپ، برخورد با دیواره، رنگ آجرها و شنیدن کلید Space، همه در یک جا. با اولین خطای پنهان، پیدا کردن علت خیلی سخت می‌شود. برای جلوگیری از این وضعیت، الگوی مدل-نما-کنترل‌گر[۱۶] وظایف را میان سه بخش مستقل تقسیم می‌کند:

لایه مدل Model | کلاس‌های منطق

وضعیت واقعی بازی: مختصات توپ، سکو و آجرها و تعداد جان‌ها. این کلاس‌ها به پنجره و گرافیک وابسته نیستند و آزمودنشان ساده است.

لایه نما BreakoutCanvas | View

ترسیم وضعیت بازی روی صفحه: اشکال، آجرها و نشانگر جان‌ها. این لایه وضعیت را از کنترل‌گر می‌گیرد و هیچ منطق بازی ندارد.

لایه کنترل‌گر GameEngine | Controller

هماهنگ‌کنندهٔ بازی: حلقهٔ بازی را ۶۰ بار در ثانیه اجرا می‌کند، فشردن کلیدها را دریافت می‌کند، برخوردها را حل می‌کند و به نما می‌گوید دوباره رسم کند.

مسیر داده در سه گام

۱. **ورودی:** کنترل‌گر کلیدهای جهت، Space، P و R را دریافت می‌کند.
۲. **محاسبه:** کنترل‌گر متدهای حرکت سکو و توپ را صدا می‌زند و وضعیت تازهٔ بازی را در مدل ثبت می‌کند.
۳. **نمایش:** پس از پایان محاسبات فیزیک، کنترل‌گر وضعیت تازه را برای رسم به نما می‌دهد.

قرارداد ذخیره رکورد: HighScoreStorage

بازی نباید به نحوهٔ ذخیرهٔ رکورد وابسته باشد. برای همین رابط ساده‌ای تعریف می‌کنیم. در فاز اول، ذخیره در حافظه کافی است. ذخیره در فایل در همین فاز **اختیاری** است؛ اگر آن را انجام دهید، در بخش امتیازی نمره می‌گیرید و برای فاز بعد و مبحث ورودی/خروجی هم مفید است:

```
public interface HighScoreStorage {
    int loadHighScore();
    void saveHighScore(int score);
}
```

پیاده‌سازی موقت در حافظه InMemoryScoreStorage

بالاترین رکورد را در حافظهٔ موقت[۱۷] نگه دارید؛ با بستن برنامه از بین می‌رود. این پیاده‌سازی برای تحویل فاز اول الزامی است.

ذخیره در فایل FileScoreStorage

ذخیرهٔ دائمی رکورد در یک فایل متنی. می‌توانید همین فاز آن را انجام دهید یا به فاز بعد بسپارید؛ در هر دو حالت رابط HighScoreStorage تغییر نمی‌کند.

---

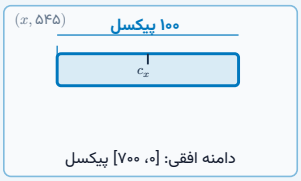
[۱۶] MVC: Model-View-Controller
[۱۷] RAM: Random Access Memory

موجودیت‌ها و هندسه بازی

بازی Breakout از چهار موجودیت تشکیل شده است. هر کدام ابعاد مشخص دارند و داده‌های داخلی‌شان پنهان است. مختصات از مبدأ $(۰, ۰)$ در گوشهٔ بالا-چپ بوم شروع می‌شود و همهٔ محاسبات بر حسب پیکسل است:

سکو `Paddle`

ابعاد ۱۰۰×۱۵ پیکسل | Y=۵۴۵ پیکسل | بازه X: [۰, ۷۰۰]

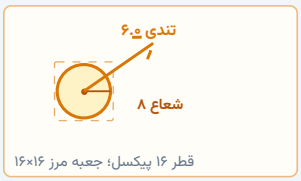


هندسه و ابعاد: مستطیل ۱۰۰ در ۱۵ پیکسل که ۴۰ پیکسل بالاتر از کف پنجره قرار دارد ($y = ۵۴۵$). سکو فقط در بازهٔ $x \in [۰, ۷۰۰]$ جابه‌جا می‌شود تا از لبه‌های بوم بیرون نزند.

رفتار و کپسوله‌سازی: فیلدهای مختصات `private` هستند. سکو فقط با `moveLeft` و `moveRight` جابه‌جا می‌شود، با سرعت ۸ پیکسل در هر تیک و با کنترل مرز.

توپ `Ball`

قطر ۱۶ | شعاع ۸ | تندی پایه ۶.۰ پیکسل/تیک

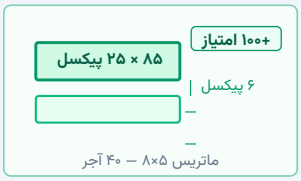


هندسه و ابعاد: دایره‌ای به قطر ۱۶ پیکسل. موقعیت آن گوشهٔ بالا-چپ جعبهٔ مرزی ۱۶×۱۶ است. با دیواره‌ها و آجرها، `reverseX` و `reverseY` مؤلفهٔ سرعت را برمی‌گردانند. بازتاب از سکو زاویه‌دار است و در بخش فیزیک توضیح داده شده است؛ بازتاب آینه‌ای ساده کافی نیست.

رفتار و کپسوله‌سازی: در شروع هر دور توپ روی سکو قرار دارد؛ مثلاً در $(x_{\text{paddle}} + ۴۲, ۵۲۹)$، یعنی وسط سکو منهای شعاع توپ. با Space پرتاب می‌شود. فیلدهای سرعت و موقعیت `private` هستند. برای تغییر سرعت از سکو، از `setVx`/`setVy` یا متد `bounceOffPaddle` استفاده کنید.

آجرها `Brick`

ماتریس ۴۰ آجر (۵×۸) | ابعاد ۸۵×۲۵ پیکسل | فاصله ۶ پیکسل

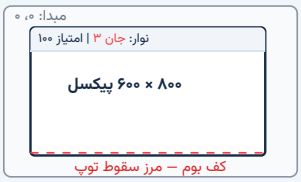


هندسه و ابعاد: ۴۰ آجر در ۵ سطر و ۸ ستون. هر آجر ۸۵ در ۲۵ پیکسل است و تا آجر بعدی ۶ پیکسل فاصله دارد. حاشیهٔ بالای شبکه ۶۰ پیکسل است. مکان آجر سطر r و ستون c از این فرمول به دست می‌آید: $x = ۳۹ + c \times ۹۱$ و $y = ۶۰ + r \times ۳۱$.

رفتار و کپسوله‌سازی: هر آجر یک مستطیل مرزی[۱۸] و وضعیت تخریب `private` دارد. متد `hit` با اصابت، آجر را از بین می‌برد و ۱۰۰ امتیاز ثبت می‌کند.

نشانگر وضعیت و بوم `HUD / Canvas`

بوم ۸۰۰×۶۰۰ پیکسل | نوار وضعیت ارتفاع ۳۵ پیکسل



هندسه و ابعاد: پنجرهٔ بازی ۸۰۰ در ۶۰۰ پیکسل است و مبدأ $(۰, ۰)$ در گوشهٔ بالا-چپ قرار دارد. نوار بالای صفحه ۳۵ پیکسل ارتفاع دارد و جان‌ها، امتیاز و رکورد را نشان می‌دهد.

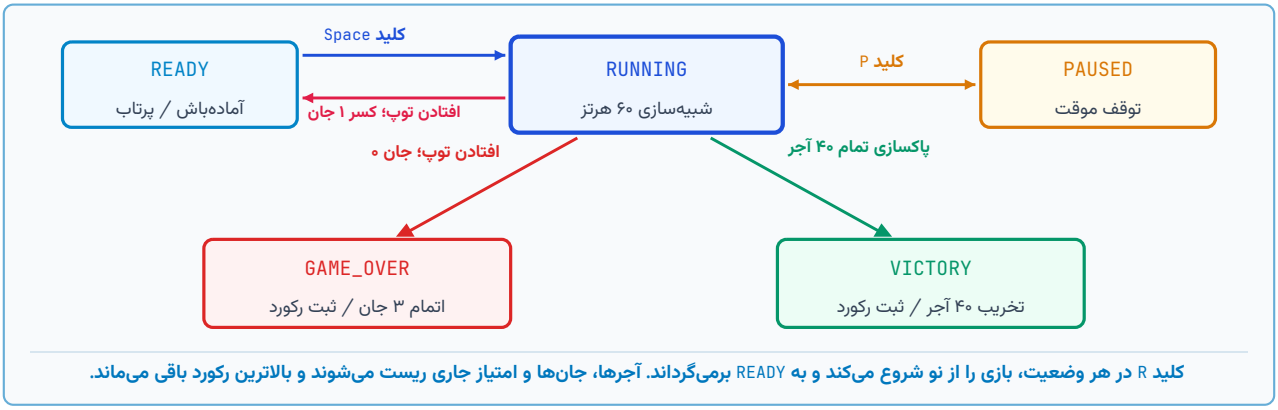
رفتار و کپسوله‌سازی: موتور بازی جان‌ها و امتیاز را نگه می‌دارد. وقتی لبهٔ پایین توپ از خط پایین بوم رد شود، یک جان کم می‌شود؛ شرط دقیق در بخش گیم‌پلی آمده است. با تمام شدن جان‌ها یا تخریب ۴۰ آجر، نما پیام پایان را نشان می‌دهد.

---

[۱۸] AABB: Axis-Aligned Bounding Box

سازوکار گیم‌پلی و وضعیت‌ها

جریان بازی با یک ماشین وضعیت متناهی پیاده‌سازی می‌شود. واکنش بازی به کلیدها، برخوردها و پایان بازی بر اساس وضعیت فعلی تعیین می‌شود:



کلید R در هر وضعیت، بازی را از نو شروع می‌کند و به READY برمی‌گرداند. آجرها، جان‌ها و امتیاز جاری ریست می‌شوند و بالاترین رکورد باقی می‌ماند.

وضعیت	رفتار، شرایط ورود و انتقال‌های مجاز در جریان بازی
حالت آماده‌باش READY	وضعیت آغاز بازی، و وضعیتی که بعد از افتادن توپ به آن برمی‌گردیم (اگر جان مانده باشد). توپ روی مرکز سکو می‌ماند و همراه سکو حرکت می‌کند. فقط ورودی سکو پردازش می‌شود. با فشردن Space، شبیه‌سازی شروع می‌شود و بازی به RUNNING می‌رود.
حالت اجرای فعال RUNNING	حلقهٔ اصلی بازی با ۶۰ هرتز اجرا می‌شود: توپ حرکت می‌کند، سکو به کلیدها واکنش نشان می‌دهد و توپ از دیواره‌ها و آجرها برمی‌گردد. اگر لبهٔ پایین توپ از خط پایین بوم رد شود، یک جان کم می‌شود. اگر جان مانده باشد، بازی به READY برمی‌گردد. با تخریب همهٔ آجرها، وضعیت VICTORY فعال می‌شود.
توقف موقت PAUSED	با P، محاسبات متوقف می‌شوند و پیام توقف نمایش داده می‌شود. دوباره P را بزنید تا بازی از همان مکان و با همان سرعت ادامه پیدا کند.
پایان بازی / شکست GAME_OVER	لبهٔ پایین توپ از خط پایین بوم رد شود و جانی نمانده باشد. شبیه‌سازی متوقف می‌شود، پیام پایان نمایش داده می‌شود و اگر امتیاز از بالاترین رکورد بیشتر باشد، رکورد ثبت می‌شود. کلید R بازی را ریست می‌کند.
پیروزی بازیکن VICTORY	با تخریب همهٔ ۴۰ آجر فعال می‌شود. توپ می‌ایستد، پیام تبریک و امتیاز نهایی نمایش داده می‌شود و بالاترین رکورد ذخیره می‌شود. کلید R یک بازی تازه را شروع می‌کند.

پیاده‌سازی تمیز وضعیت‌ها با ساختار شمارشی enum

به جای چند متغیر بولی مثل isPaused یا isGameOver، وضعیت بازی را با enum GameState و ۵ مقدار بالا نشان دهید. در READY فقط سکو و توپ چسبیده به آن به‌روز می‌شوند. فیزیک و برخورد فقط در RUNNING اجرا می‌شوند. این جداسازی جلوی بسیاری از باگ‌های منطقی را می‌گیرد.

گرافیک و واسط کاربری

نمایش Breakout روی یک بوم دوبعدی انجام می‌شود. برای اینکه تصویر روان و بدون لرزش باشد، چند نکته را رعایت کنید:

۱. دستگاه مختصات و هندسه بوم

در گرافیک کامپیوتری جهت محورها با مختصات ریاضی فرق دارد:

- مبدأ $(۰, ۰)$ در گوشهٔ بالا-چپ بوم است.
- محور x به سمت راست و مثبت است.
- محور y به سمت پایین و مثبت است.

پس وقتی توپ به سمت بالا و آجرها می‌رود، سرعت عمودی‌اش منفی است و وقتی به طرف سکو سقوط می‌کند، مثبت است.

۲. پایداری تصویر و بافر دوگانه

اگر در برنامه‌های گرافیکی تصویر چشمک بزند یا بلرزد، دلیلش پاک و دوباره کشیده شدن پی‌درپی اشیا روی صفحه است:

- با بافر دوگانه[۱۹]، همهٔ نقاشی‌ها ابتدا در حافظهٔ پشت‌صحنه کامل می‌شوند و بعد یک‌جا روی صفحه نشان داده می‌شوند.
- کتابخانهٔ ACM این کار را خودش انجام می‌دهد، پس برای ساخت انیمیشن روان لازم نیست نگرانش باشید.

۳. اجزای دیداری و نوار وضعیت

بخش بصری	مشخصات ترسیم
بوم اصلی بازی	مستطیل ۸۰۰ در ۶۰۰ پیکسل با پس‌زمینهٔ تیره یا یکدست که محدودهٔ بازی را مشخص می‌کند.
سکو و توپ	سکو به صورت مستطیلی توپر با رنگ متمایز و توپ به شکل دایره‌ای یکدست با قطر ۱۶ پیکسل.
شبکه آجرها	مستطیل‌های هم‌اندازه با فاصلهٔ ۶ پیکسل. رنگ سطرها را آزاد انتخاب کنید.
نوار وضعیت[۲۰]	نوار بالای صفحه با ارتفاع ۳۵ پیکسل برای امتیاز، ۳ جان و رکورد. پیام‌های توقف و پایان وسط بوم نمایش داده می‌شوند.

[۱۹] Double Buffering
[۲۰] HUD: Heads-Up Display

فیزیک و برخوردها

موتور فیزیک حرکت توپ، زاویهٔ بازتاب از سکو و تخریب آجرها را محاسبه می‌کند. هر «تیک» یک بار اجرای حلقهٔ ۶۰ هرتزی بازی است:

۱. بازتاب زاویه‌دار از سکو

اگر توپ مثل آینه بازتاب کند، بازی خیلی زود یکنواخت می‌شود. برای اینکه هر ضربه متفاوت باشد، زاویهٔ بازتاب به فاصلهٔ نقطهٔ برخورد از مرکز سکو بستگی دارد:

روابط ریاضی کمانه زاویه‌دار

- **۱. انحراف برخورد نسبت به مرکز سکو ($d \in [-1.0, +1.0]$):** فاصلهٔ افقی مرکز توپ تا مرکز سکو (Δx) و نصف عرض سکو (w_{half}) را جدا تعریف می‌کنیم:

$$\Delta x = x_{\text{ball}} - x_{\text{paddle_center}} \quad , \quad w_{\text{half}} = \frac{W_{\text{paddle}}}{2}$$

$$d = \frac{\Delta x}{w_{\text{half}}}$$

Δx منفی یعنی توپ به نیمهٔ چپ سکو خورده و Δx مثبت یعنی به نیمهٔ راست.

- **۲. زاویهٔ بازتاب (حداکثر ۶۰ درجه):**

$$\theta = d \times 60^{\circ}$$

اصابت به مرکز سکو بازتاب عمودی ($\theta = 0$) می‌دهد. اصابت به لبه‌ها ($d = \pm 1.0$) بیشترین زاویه، یعنی ۶۰ درجه، را می‌دهد.

- **۳. حفظ تندی کل توپ:** اندازهٔ سرعت را با قضیهٔ فیثاغورس حساب کنید:

$$v = \sqrt{v_x^2 + v_y^2}$$

- **۴. تجزیهٔ سرعت به مؤلفه‌های جدید (توپ به بالا برمی‌گردد):**

$$v_{x'} = v \sin(\theta) \quad , \quad v_{y'} = -|v \cos(\theta)|$$

علامت منفی در مؤلفهٔ عمودی باعث می‌شود توپ همیشه به سمت بالا، یعنی به طرف آجرها، برگردد.

این بخش فقط بازتاب از سکو را پوشش می‌دهد. برای دیواره‌ها و آجرها از `reverseX` و `reverseY` استفاده کنید.

```
// Calculate horizontal offset and half-width as separate variables
double paddleCenter = paddle.getX() + paddle.getWidth() / 2.0;
double ballCenter = ball.getX() + ball.getRadius();
double deltaX = ballCenter - paddleCenter;
double halfWidth = paddle.getWidth() / 2.0;

double offset = deltaX / halfWidth;
offset = Math.max(-1.0, Math.min(1.0, offset));

// Calculate bounce angle (max 60 deg) and preserve total velocity
double angle = offset * Math.toRadians(60.0);
double speed = Math.hypot(ball.getVx(), ball.getVy());

// Apply new velocity components (always deflect upwards)
ball.setVx(speed * Math.sin(angle));
ball.setVy(-Math.abs(speed * Math.cos(angle)));
```

۲. تشخیص برخورد با آجرها

برای تشخیص برخورد توپ با آجرها و دیواره‌ها از الگوریتم مستطیل‌های مرزی[۲۱] استفاده کنید. هنگام برخورد، عمق هم‌پوشانی توپ و آجر را در محور افقی و عمودی حساب کنید و فقط مؤلفهٔ سرعتی را معکوس کنید که عمق کمتری دارد. در گوشهٔ آجر هم همین قاعده را اعمال کنید تا فقط یک مؤلفه برگردد. با نابودی هر آجر ۱۰۰ امتیاز ثبت می‌شود.

[۲۱] AABB: Axis-Aligned Bounding Box

نکته‌هایی برای پیاده‌سازی درست فیزیک

۱. **توپ داخل سکو گیر می‌کند:** اگر فقط علامت سرعت را عوض کنید، ممکن است توپ در فریم بعد هنوز داخل سکو باشد و بلرزد. بعد از برخورد، موقعیت عمودی توپ را طوری تنظیم کنید که لبهٔ پایین جعبهٔ مرز توپ دقیقاً به لبهٔ بالای سکو برسد؛ برای مثال `ball.setY(paddle.getY() - 2 * ball.getRadius())` وقتی `getY` گوشهٔ بالا-چپ را برمی‌گرداند.

۲. **نتیجهٔ صفر در تقسیم صحیح:** برای نسبت فاصله‌ها از نوع اعشاری `double` استفاده کنید. در غیر این صورت تقسیم صحیح، نتیجه را صفر می‌کند.

۳. **عبور توپ از آجر[۲۲]:** اگر توپ در یک تیک خیلی دور برود، ممکن است از روی آجر بپرد و برخورد ثبت نشود. در هر تیک توپ را کمتر از قطرش (۱۶ پیکسل) جابه‌جا کنید.

[۲۲] Tunneling

گام‌های پیاده‌سازی

پیشنهاد می‌کنیم ساخت بازی را در پنج گام پیش ببرید:

۱. **گام اول: پنجره، بوم و سکوی بازی**
پنجرهٔ ۸۰۰ در ۶۰۰ پیکسل را بسازید، کلاس سکو را بنویسید و حرکت افقی آن را با کلیدهای چپ و راست کنترل کنید.
معیار پذیرش: سکو روان حرکت کند و در لبهٔ چپ و راست صفحه بایستد.

۲. **گام دوم: کلاس توپ، حرکت و بازتاب از دیواره‌ها**
کلاس توپ و بردار سرعت آن را بسازید و توپ را از سه دیوارهٔ بالا، چپ و راست بوم بازتاب دهید.
معیار پذیرش: توپ در ابتدا همراه سکو حرکت کند و با فشردن `Space` پرتاب شود.

۳. **گام سوم: بازتاب زاویه‌دار از سکو و سیستم ۳ جان**
بازتاب زاویه‌دار از سکو با زاویه‌ای بین منفی ۶۰ و مثبت ۶۰ درجه، و کم شدن یک جان وقتی لبهٔ پایین توپ از خط پایین بوم عبور کند.
معیار پذیرش: با از دست رفتن توپ یک جان کم شود و توپ دوباره روی سکو قرار گیرد.

۴. **گام چهارم: ماتریس ۴۰ آجر، تشخیص برخورد و امتیاز**
۴۰ آجر را در ۵ سطر و ۸ ستون بچینید، الگوریتم برخورد را بنویسید، برای هر آجر تخریب‌شده ۱۰۰ امتیاز بدهید و بالاترین رکورد را در حافظهٔ موقت نگه دارید.
معیار پذیرش: با اصابت توپ، آجر حذف شود و جهت توپ تغییر کند.

۵. **گام پنجم: ماشین وضعیت، توقف، برد، باخت و شروع دوباره**
وضعیت‌های بازی را به هم وصل کنید، با کلید `P` پیام توقف نشان دهید و با کلید `R` بازی را در هر لحظه از نو شروع کنید.
معیار پذیرش: بازی با از دست رفتن همهٔ جان‌ها یا تخریب همهٔ آجرها تمام شود، کلید `R` بازی را ریست کند و بالاترین رکورد باقی بماند.

راهبرد توسعه و کامیت‌ها

بعد از هر گام یک کامیت با پیشوند استاندارد (مثلاً `feat:`) در گیت‌هاب ثبت کنید. با این کار همیشه یک نسخهٔ سالم از پروژه در تاریخچه دارید.

معیارهای ارزیابی و ارائه

فاز اول از ۱۰۰ نمره تشکیل شده است که معادل ۲ نمره از ۲۰ نمرهٔ پایانی درس است. نمرهٔ امتیازی اختیاری حداکثر ۱۵ نمره است و جدا از این ۱۰۰ نمره حساب می‌شود:

معیار ارزیابی	مؤلفه‌ها و جزئیات بررسی فنی	بارم
درستی عملکرد و حس بازی	حرکت روان ۶۰ هرتز، پرتاب با Space، بازتاب زاویه‌دار از سکو، بازتاب الاستیک از سه دیواره، تخریب درست آجرها با الگوریتم مستطیل‌های برخورد، چرخهٔ ۳ جان و ماشین وضعیت کامل.	۳۵ نمره
معماری و طراحی شی‌گرا	جدا بودن سه لایهٔ مدل، نما و کنترل‌گر، کپسوله‌سازی فیلدها با دسترسی private، رعایت اصل تک‌مسئولیتی و استفاده از رابط HighScoreStorage.	۳۵ نمره
کیفیت کد و مهندسی نرم‌افزار	بدون عددهای جادویی (ثابت‌ها در یک کلاس جداگانه)، نام‌گذاری روشن متغیرها و توابع، ساختار استاندارد پوشه‌ها و بسته‌ها و کامیت‌های منظم در گیت‌هاب.	۱۵ نمره
تسلط و گفت‌وگو در جلسه ارائه	توضیح محاسبات زاویهٔ بازتاب، منطق تشخیص برخورد و تصمیم‌های شی‌گرایی، پاسخ به پرسش‌های مفهومی و در صورت خواسته شدن، اعمال یک تغییر کوچک در کد.	۱۵ نمره
مجموع کل ارزیابی	**ارزیابی کلی هستهٔ بازی Breakout در فاز اول**	**۱۰۰ نمره**

یادداشت برای ارزیابان

در پیاده‌سازی مرجع، توپ وقتی از دست می‌رود که لبهٔ پایین جعبهٔ مرزی ۱۶×۱۶ از خط $y = 600$ (لبهٔ پایین بوم ۸۰۰×۶۰۰) عبور کند. این همان شرط بخش گیم‌پلی است. موقعیت توپ با گوشهٔ بالا-چپ ثبت می‌شود. پیاده‌سازی‌هایی که همین هندسه را دارند، پذیرفته می‌شوند.

جزئیات جلسه ارائه

جلسهٔ ارائه بعد از تحویل مخزن برگزار می‌شود. هدف آن نشان دادن کار شما و گرفتن بازخورد است. اگر کد را خودتان نوشته‌اید و دلیل تصمیم‌هایتان را می‌دانید، جلسه راحت پیش می‌رود:

- **تسلط بر کد:** اگر ساختار کد را بشناسید و منطق فیزیک و کلاس‌ها را روان توضیح دهید، می‌توانید ۱۵ نمرهٔ این ردیف را کامل بگیرید.
- **استقلال کار:** هدف این جلسه مطمئن شدن از یادگیری واقعی شماست. اگر نتوانید دلیل تصمیم‌های کدتان را توضیح دهید یا در جلسه تغییر کوچکی را اعمال نکنید، نمرهٔ فنی با ضریب α کم می‌شود. بعد از جلسه، دستیاران میزان تسلط شما را با عددی بین ۰ و ۱۰۰ (یعنی m) می‌سنجند و ضریب را این‌طور حساب می‌کنند: $\alpha = 0.5 + 0.5 \cdot \frac{m}{100}$. نمرهٔ نهایی فاز برابر است با نمرهٔ فنی ضربدر α. مثلاً اگر نمرهٔ فنی ۸۰ و میزان تسلط ۶۰ باشد، آنگاه $\alpha = ۰/۸$ و نمرهٔ نهایی ۶۴ می‌شود. با این فرمول همواره $0.5 \le \alpha \le 1$ است: تسلط صفر یعنی ضریب ۰٫۵ و تسلط کامل یعنی ضریب ۱.

بخش‌های امتیازی و اختیاری

دو قابلیت اختیاری زیر نمره دارند. مجموع نمرات امتیازی حداکثر ۱۵ است و جزو ۱۰۰ نمرهٔ پایه حساب نمی‌شود:

- **آجرهای مقاوم چندضربه‌ای — ۵ نمره امتیازی:** آجر با ضربهٔ اول آسیب می‌بیند و با ضربهٔ بعدی از بین می‌رود. امتیاز فقط وقتی داده می‌شود که آجر کاملاً از بین برود، نه با هر اصابت.
- **تعویض پویای راهبرد ذخیره‌ساز — ۱۰ نمره امتیازی:** در حین اجرای بازی بتوان نوع ذخیرهٔ رکورد را بین حافظهٔ موقت و فایل متنی تغییر داد.